

GIT VERSION CONTROL

Git Commands

شرح عملي ومفهوم لأوامر Git مع أمثلة وسيناريوهات من الواقع

Tariq Zeyad

Practical Git Commands Research



مقدمة

لما نشتغل على مشروع برمجي، خصوصًا إذا المشروع كبير أو أكثر من شخص يشتغل عليه، بصير مهم نعرف كيف نحافظ على تاريخ المشروع وكيف نرجع للتغييرات القديمة وكيف نتعامل مع الفروع المختلفة.

Git هو نظام Version Control يساعدنا ندير تاريخ المشروع والتغييرات اللي بتصير عليه. لكن Git مش مجرد أوامر للحفظ والرفع؛ كل أمر فيه بحل مشكلة معينة.

في هذا البحث رح أشرح مجموعة من أوامر Git بطريقة عملية. الفكرة مش إني أحفظ الأمر فقط، وإنما أفهم شو المشكلة اللي بحلها ومتى ممكن أحججه في مشروع حقيقي.

المحتويات

1. Git Squash

2. Git Merge vs Git Rebase

3. Git Help

4. Git Cherry-pick

5. Git Clean

6. Git Grep

7. Git Blame

8. Git Bisect

9. Git Shortlog

10. Git Prune

11. Git Worktree

12. Git Verify-commit

13. Git Filter-repo

Git Squash

01

شو يعني Squash؟

أول نقطة لازم تكون واضحة: ما في أمر مستقل اسمه `git squash`. لما نحكي عن Squash في Git، غالبًا بنقصد خيار `squash` اللي بنستخدمه داخل Interactive Rebase.

فكرته بسيطة: إذا عملت أكثر من Commit أثناء تطوير Feature وكل الـ Commits بالنهاية متعلقين بنفس الشغلة، بقدر أدمجهم في Commit واحد.

ليش أحتاجه؟

أثناء الشغل عادي جدًا أعمل Commits صغيرة، مثل:

```
Add Login Button
Fix Login Button
Add Username Validation
Fix Validation
Add Password Validation
```

هاي الـ Commits مفيدة أثناء التطوير، لكن لما أخلص الـ Feature ممكن أحب يكون تاريخ المشروع مرتب وواضح أكثر.

مثال من الواقع

افترض إنني خلصت Login Feature وعندي آخر 3 Commits مرتبطين فيها.

أول إشي بشوف آخر 3 Commits:

```
git rebase -i HEAD~3
```

رح يظهر شيء قريب من:

```
pick 4100796 Add Login Button
squash 1e4f96e Add Username Validation
squash dbb03b0 Add Password Validation
```

هون أول Commit رح يضل، والـ Commit الثاني والثالث رح يندمجوا معه.

شو النتيجة؟

بدل ما يكون عندي 3 Commits منفصلين، بصير عندي Commit واحد يحتوي التغييرات الثلاثة.

انتبه:

Squash بغير تاريخ الـ Commits، وبالتالي ممكن تتغير Commit IDs. إذا كانت الـ Commits مرفوعة على Remote والناس الثانية شغالة عليها، لازم تنتبه قبل ما تعمل Rebase أو Squash.

Git Merge vs Git Rebase

02

Git Merge

Merge بنستخدمه لما يكون عندي Branch وبدي أدمجه مع Branch ثاني.

```
git switch main
git merge feature-login
```

هون Git بحاول يدمج التغييرات الموجودة في `feature-login` داخل `main`.

مثال من الواقع

أنا وفريقي شغالين على مشروع. أنا عملت Feature اسمها Login، وخلصت منها. هسا بدي أدمجها مع Main.

```
git switch main
git merge feature-login
```

إذا كان فيه اختلافات بين الفرعين، Git ممكن يعمل Merge Commit أو يستخدم Fast-forward حسب حالة التاريخ.

Git Rebase

Rebase فكرته مختلفة. بدل ما أعمل Merge، بقدر أخلي الـ Commits تبعتي تنعاد فوق آخر نسخة من Branch معين.

```
git switch feature-login
git rebase main
```

كأنني يقول Git: "خذ شغلي وحطه فوق آخر نسخة موجودة في Main."

مثال من الواقع

أنا شغال على Feature من يومين، وخلال هاليومين الفريق أضاف Commits جديدة على Main.

بقدر أعمل:

```
git switch feature-login
git rebase main
```

هيك بتصير Feature مبنية فوق أحدث نسخة من Main.

الفرق بشكل سريع

Rebase	Merge
بعيد تطبيق الـ Commits فوق قاعدة جديدة.	بدمج تاريخ فرعين.
غالبًا يعطي تاريخ خطي وأنظف.	ممكن ينشئ Merge Commit.
لازم ننتبه إذا الـ Commits منشورة ومستخدم من الآخرين.	مناسب جدًا للفروع المشتركة.

فكر فيها هيك:

Merge: "خلينا نجمع التاريخين."

Rebase: "خليني أعيد ترتيب شغلي كأنه مبني من النقطة الجديدة."

شو بعمل؟

إذا نسيت كيف تستخدم أمر معين، Git نفسه عنده نظام مساعدة. يعني مش لازم تحفظ كل Option موجود في Git.

```
git help rebase
```

هذا بفتح Documentation الخاصة بـRebase.

وممكن مثلاً:

```
git help commit
```

مثال من الواقع

افتترض إنني بدي أعمل Rebase، بس نسيت خيار معين داخل الأمر. بدل ما أجرب عشوائي، يكتب:

```
git help rebase
```

وبراجع الخيارات المتاحة والطريقة الصحيحة للاستخدام.

طريقة ثانية

```
git rebase --help
```

الفكرة:

مش لازم تحفظ كل شيء. المهم تعرف كيف ترجع للتوثيق الصحيح لما تحتاج معلومة.

شو بعمل؟

Cherry-pick بخليني آخذ Commit معين وأطبق التغييرات الموجودة فيه على الـ Branch اللي أنا واقف عليه. المميز فيه إنني مش مضطر أجيب الـ Branch كامل. أنا بختار Commit واحد بالذات.

```
git cherry-pick 8a42f91
```

مثال من الواقع

أنا شغال على Main، وفي Branch اسمه Feature Login. الـ Feature Login لسا مش جاهزة عشان أدمجها كلها. لكن فيها Commit واحد فيه إصلاح مهم. أنا بدي الإصلاح فقط.

```
git switch main  
git cherry-pick 8a42f91
```

هيك Git بأخذ التغيير الموجود في هذا الـ Commit ويطبقه على Main كـ Commit جديد.

إذا صار Conflict

إذا Git ما قدر يطبق التغيير مباشرة، ممكن يصير Conflict. بعد ما أحل المشكلة:

```
git add .  
git cherry-pick --continue
```

وإذا قررت ألغي العملية:

```
git cherry-pick --abort
```

Cherry-pick = "أنا بدي هذا Commit بالذات، مش كل الBranch".

Git Clean

05

شو بعمل؟

أثناء الشغل ممكن أعمل ملفات مؤقتة أو ملفات للتجربة. إذا Git مش متتبع هاي الملفات، ممكن أستخدم `git clean` لإزالتها.

أول خطوة: افحص

```
git clean -n
```

هذا الأمر ما بحذف شيء. فقط بفرجيني الملفات اللي ممكن تنحذف.

الحذف

```
git clean -f
```

مثال من الواقع

خلال التجربة عملت:

```
temp.txt  
debug.log  
test.txt
```

هنا الملفات مش جزء من المشروع ومش متتبعة. أول شيء:

```
git clean -n
```

إذا تأكدت إنها فعلاً ملفات مؤقتة:

```
git clean -f
```

انتبه:

الملفات غير المتتبعة اللي بحذفها Git Clean ممكن ما تقدر ترجعها من Git، لأنها أصلاً مش موجودة ضمن تاريخ Git.

عشان هيك دائماً ابدأ بـ `git clean -n`.

Git Grep

06

شو بعمل؟

إذا المشروع كبير وفيه كثير ملفات، والبحث اليدوي صار مزعج، `git grep` بخليني أدور على كلمة أو نص داخل الملفات اللي بتتبعها.

```
git grep "isAuthenticated"
```

مع رقم السطر

```
git grep -n "isAuthenticated"
```

مثال من الواقع

عندي مشروع Kotlin كبير، وبدي أعرف وين مستخدمة الدالة:

```
isAuthenticated()
```

بدل ما أفتح عشرات الملفات:

```
git grep "isAuthenticated"
```

Git بطلعلي الأماكن اللي فيها النص.

فكر فيها هيك:

Git Grep = "دورلي على هذا النص داخل المشروع."

شو بعمل؟

Blame يساعدني أعرف أي Commit قدم النسخة الحالية لكل سطر في ملف معين، ومين الـ Author المرتبط بهذا Commit.

```
git blame App.java
```

مثال من الواقع

عندي سطر بالكود عامل مشكلة. أنا مش عارف متى انضاف أو بأي Commit تغير.

```
git blame App.java
```

بعدها بقدر آخذ Commit ID وأشوف تفاصيله:

```
git show <commit-id>
```

فحص جزء معين من الملف

```
git blame -L 20,35 App.java
```

هون بفحص الأسطر من 20 إلى 35 فقط.

ملاحظة مهمة:

اسم الأمر Blame مش معناه إن الشخص هو المسؤول عن الخطأ. هو فقط طريقة لمعرفة الـ Commit والـ Author المرتبطين بالنسخة الحالية من السطر.

شو المشكلة اللي بحلها؟

تخيل المشروع كان شغال تمام قبل أسبوع، واليوم صار فيه Bug. أنت عارف إنه في وقت معين دخلت المشكلة، لكن مش عارف بأي Commit.

إذا عندك 100 Commit، فحصهم واحد واحد رح يكون متعب جدًا. Git Bisect يستخدم Binary Search عشان يقلل عدد الCommits اللي لازم تفحصها.

نبدأ العملية

```
git bisect start
```

بنقول لـ Git إن النسخة الحالية فيها المشكلة:

```
git bisect bad
```

وبعدين بنعطيه Commit قديم إحنا متأكدين إنه كان شغال:

```
git bisect good 4994b1f
```

Git رح ينقلنا إلى Commit في منتصف الفترة تقريبًا. بنجرب البرنامج ونقرر.

إذا المشكلة موجودة:

```
git bisect bad
```

وإذا المشكلة مش موجودة:

```
git bisect good
```

مثال من الواقع

عندي 100 Commit، والبرنامج كان شغال في البداية، وحاليًا فيه Bug.

Git ما بفحصهم كلهم. يبدأ من النص تقريبًا، وأنا كل مرة بقله إذا النسخة فيها المشكلة أو لا.

مع كل إجابة، Git بقلل نطاق البحث لحد ما يوصل للCommit اللي دخل المشكلة.

بعد ما أخلص

```
git bisect reset
```

ببساطة:

Git Bisect = "ساعدني أعرف بأي Commit انكسر المشروع."

Git Shortlog

09

شو بعمل؟

Shortlog بعمل ملخص لتاريخ Commits وجمعها حسب الأشخاص اللي عملوها.

```
git shortlog
```

ممکن يطلع شيء مثل:

Ahmed (10):

Add Login

Fix Login

Tariq (7):

Add Routing

Fix Tests

عدد الCommits فقط

```
git shortlog -sn
```

ممکن يعطی:

```
15    Ahmed
```

```
10    Tariq
```

```
7     Omar
```

مثال من الواقع

خلصنا Sprint وبدي أعمل ملخص سريع عن مساهمات أعضاء الفريق. بقدر أستخدم:

```
git shortlog -sn --all
```

وهيك أشوف عدد الCommits حسب الAuthors.

شو يعني Prune؟

Git داخليًا بخزن بيانات المشروع على شكل Objects، مثل Blobs و Trees و Commits. أحيانًا بعد إعادة كتابة التاريخ أو حذف References، بصير عندنا Objects ما عاد في Reference يوصل إلها. هاي Objects ممكن تظل موجودة لفترة، وهون بييجي دور عمليات التنظيف مثل Prune.

```
git prune
```

مثال من الواقع

عملت Rebase وغيّرت تاريخ المشروع، وبعد العملية صار فيه Objects قديمة ما عاد في Branch أو Reference يوصل إلها. Git عنده آليات Garbage Collection وصيانة بتتعامل مع البيانات غير المستخدمة، و Prune جزء منخفض المستوى من هذه العملية.

لا تخلط بين Clean و Prune

الأمر	شو بتعامل معه؟
<code>git clean</code>	ملفات غير متتبعة في Working Directory.
<code>git prune</code>	Git Objects غير القابلة للوصول.

انتبه:

Prune أمر منخفض المستوى ومش أمر عادي نستخدمه كل يوم. عشان هيك لازم نكون فاهمين شو رح ينحذف قبل تشغيله.

Git Worktree

11

شو المشكلة اللي بحلها؟

عادةً Repository واحد بيكون عنده Working Directory واحدة. يعني غالبًا يكون شغال على Branch واحد في نفس المكان.

لكن أحيانًا يكون شغال على Feature وفجأة لازم أصلح Bug على Branch ثاني.

بدل ما أعمل Stash وأغير Branch كل شوي، ممكن أستخدم Worktree.

عرض Worktrees

```
git worktree list
```

إنشاء Worktree

```
git worktree add ../project-hotfix main
```

مثال من الواقع

عندي:

```
project/  
feature-login
```

وأنا شغال على Feature Login. فجأة وصل Bug لازم يتصلح على Main.

بقدر أعمل:

```
git worktree add ../project-hotfix main
```

هسا صار عندي فولدر ثاني فيه Main، بينما الفولدر الأول بضل شغال على Feature Login.

إنشاء Branch جديد مع Worktree

```
git worktree add -b hotfix-login ../project-hotfix main
```

حذف Worktree

```
git worktree remove ../project-hotfix
```

ببساطة:

Worktree بتخليني أشتغل على أكثر من Branch بنفس الوقت، وكل واحد ممكن يكون موجود في فولدر مختلف.

شو بعمل؟

Verify-commit بنستخدمه للتحقق من التوقيع الرقمي الموجود على Commit، إذا كان الCommit موقعًا.

```
git verify-commit <commit-id>
```

مثال من الواقع

تخيل شركة عندها مشروع حساس ويتطلب إن الCommits الرسمية تكون موقعة رقميًا.

بدي أتحقق من Commit معين:

```
git verify-commit abc1234
```

Git بفحص التوقيع المرتبط بالCommit.

مع Git Log

```
git log -1 --show-signature
```

الفكرة:

Verify Commit = "هل هذا الCommit موقع بتوقيع أقدر أتحقق منه؟"

شو هو؟

Git Filter-repo أداة متقدمة لإعادة كتابة تاريخ Repository. يعني مش بس بتغير النسخة الحالية من المشروع، وإنما بتقدر تعمل Filters على التاريخ نفسه.

الأداة مفيدة جدًا لما يكون عندي مشكلة قديمة في Repository وبدي أنظف التاريخ.

مثال: حذف ملف من كامل التاريخ

افترض إنني رفعت بالخطأ ملف:

```
config.env
```

وبعدها حذفته من المشروع:

```
git rm config.env  
git commit -m "Remove config file"
```

المشكلة إن الملف ممكن يظل موجودًا داخل الـ Commits القديمة.

إذا الهدف هو إعادة كتابة التاريخ وإزالة الملف من جميع النسخ التاريخية:

```
git filter-repo --path config.env --invert-paths
```

مثال من الواقع

مطور رفع ملف بالغلط يحتوي على API Key. بعدين اكتشف المشكلة وحذف الملف.

حذف الملف من آخر Commit مش معناه إن الـ API Key اختفى من تاريخ Git.

في حالات زي هاي ممكن نستخدم Filter-repo لإعادة كتابة التاريخ وإزالة الملف منه.

نقطة أمنية مهمة جدًا:

إذا Secret مثل Password أو API Key انكشف، إعادة كتابة Git History لحالها مش كافية.

لازم نعمل Rotate أو Revoke للSecret نفسه، لأنه ممكن يكون انكشف بالفعل.

كمان لازم ننتبه إن إعادة كتابة التاريخ بتغير Commit IDs ويتأثر على الأشخاص اللي عندهم نسخ من Repository.

مثال آخر

Filter-repo ممكن يستخدم أيضًا لإعادة تنظيم تاريخ المشروع. مثلاً ممكن نأخذ مجلد ونضع محتوياته تحت مجلد جديد في التاريخ:

```
git filter-repo \
  --path src/ \
  --to-subdirectory-filter my-module
```

ببساطة:

Filter-repo مش أمر يستخدمه كل يوم. هو أداة قوية للحالات اللي بدي فيها أغير تاريخ Repository نفسه أو أنظفه.

الخلاصة

14

بعد ما نفهم الأوامر، أسهل طريقة لتذكرها هي ربط كل أمر بالمشكلة اللي بحلها، مش بحفظ اسمه فقط.

الأمـر	فكرته ببساطة
Squash	بدمج عدة Commits في Commit واحد.
Merge	بدمج Branch مع Branch ثاني.
Rebase	بعيد تطبيق Commits فوق قاعدة جديدة.
Help	بساعدني أعرف طريقة استخدام أمر معين.
Cherry-pick	بجيب Commit محدد بدون دمج Branch كامل.
Clean	بنظف الملفات غير المتتبعة.
Grep	بدور على نص داخل ملفات المشروع.
Blame	بساعدني أعرف أي Commit مرتبط بكل سطر.
Bisect	بساعدني ألاقي الCommit اللي دخل Bug.
Shortlog	بعطيني ملخص عن الCommits حسب الAuthors.
Prune	بتعامل مع Git Objects غير القابلة للوصول.
Worktree	بخليني أشتغل على أكثر من Working Directory.
Verify-commit	بتحقق من توقيع Commit.
Filter-repo	بستخدمه لإعادة كتابة وتنظيف تاريخ Repository.

الفكرة الأهم:

لما أفهم المشكلة اللي كل أمر بحلها، بصير سهل علي أعرف أي أمر أستخدم حتى لو ما حفظت الSyntax حرفيًا.

المصادر

- Git Documentation — <https://git-scm.com/docs> .1
- Git Rebase — <https://git-scm.com/docs/git-rebase> .2
- Git Merge — <https://git-scm.com/docs/git-merge> .3
- Git Cherry-pick — <https://git-scm.com/docs/git-cherry-pick> .4
- Git Help — <https://git-scm.com/docs/git-help> .5
- Git Clean — <https://git-scm.com/docs/git-clean> .6
- Git Grep — <https://git-scm.com/docs/git-grep> .7
- Git Blame — <https://git-scm.com/docs/git-blame> .8
- Git Bisect — <https://git-scm.com/docs/git-bisect> .9
- Git Shortlog — <https://git-scm.com/docs/git-shortlog> .10
- Git Prune — <https://git-scm.com/docs/git-prune> .11
- Git Worktree — <https://git-scm.com/docs/git-worktree> .12
- Git Verify Commit — <https://git-scm.com/docs/git-verify-commit> .13
- Git Filter Repo — <https://github.com/newren/git-filter-repo> .14